

Intelligent Enforcement of Fine-Grained Access Control Policies for SQL Queries

Master's Thesis Defense

Hoang Nguyen Phuoc Bao

October 22, 2021

Universidad Autónoma de Madrid



ABOUT ME



- 2014-2018: Undergraduate @ VGU, Vietnam.
- 2018-2019: Software developer @ AAVN, Vietnam.
- 2019-2020: Research assistant @ VGU, Vietnam:
 - Focusing on model-driven engineering.
- 2020-2021: Master student @ UAM, Spain.

OUTLINE

1. Motivation and Background
2. A model-driven approach for FGAC Policies for SQL queries
3. Intelligently enforcing FGAC Policies for SQL queries
4. Conclusions

Motivation and Background

THE THREE-TIER ARCHITECTURE



The three-tier architecture is widely used to design applications that center around data, i.e. database-centric applications.

Certainly, security mechanisms such as privacy and access control, to protect information from being disclosed unintentionally, are needed.

ACCESS CONTROL IN DATABASE-CENTRIC APPLICATIONS

			
Security	✗	—	✓
Implementation	✗	—	—
Performance	✗	✗	✓
Maintainability	✗	✗	✓
Scalability	✗	✗	✓

As a consequence, for enforcing access control in the database-centric applications, the database layer should be favored over the application layer and presentation layer.

ACCESS CONTROL ON DATABASE LAYER

- Native support for Role-based Access Control (RBAC):
 - roles represent the responsibility in the application,
 - and permissions are restrictions of accessing data, normally on table- or column-level.
- Fine-grained Access Control (FGAC): on a finer granularity
 - access control decisions are not only static, i.e. the RBAC-permissions,
 - but also based on the current database state, i.e. row- and cell-level.

In nowadays applications, the need for specifying fine-grained access control policies are increasing.

ENFORCING FGAC ON DATABASE LAYER

Industrial recommendations: Do-It-Yourself with the available features, e.g.

- parametrized views,
- extra security-related tables,
- row-level security,
- etc.

How about their correctness? — It is a non-trivial task to prove that the implementation is *correct*, with respect to the security requirements.

From the research point of view: *How about a formal, model-driven approach to enforce FGAC on the database level?*

MODEL DRIVEN ENGINEERING

- Model-driven engineering (MDE) aims to develop software systems using *models* as the driving-force.
 - *Models* are artifacts that specify the different aspects of the intended software system.
 - *Code-generators* should then bridge the gap between models and executable code.
- MDE's purported benefits:
 - Reduce software development time.
 - Reduce software cost.
 - Improve software quality.
- MDE's lessons learned:
 - Modelling can not completely replace programming.
 - But for specific domains MDE delivers what promises.

MODEL DRIVEN SECURITY

- Model-driven security (MDS) is a specialization of model-driven engineering.
- Systems design are modelled together with their security requirements.
 - *FGAC policies*: they declare who can access which information and under which circumstances.
- Security infrastructures are directly generated from the models.

SECUREUML

- SecureUML is the 'de facto' modelling language used in MDS for specifying FGAC policies.
- In SecureUML, one can model access control decisions (also known as authorization constraints) that depend on two kinds of information, namely:
 - The assignments of user to role and of role to authorization constraints,
 - and the satisfaction of authorization constraints by the current state of the database system.
- Finally, in SecureUML, access control decisions are specified using the Object Constraint Language.

OBJECT CONSTRAINT LANGUAGE

- The Object Constraint Language (OCL) is a textual language typically used for specifying constraints and queries over UML models.
- OCL expressions are written in the context of a model, called the *contextual* model.
- OCL is strongly typed. Expressions either have a *primitive* type (boolean, integer, real, string), a *class* type, a *tuple* type, a *collection* type, or a *special* type.

OBJECT CONSTRAINT LANGUAGE (CONTINUED)

- It provides a *dot*-operator to access the values of attributes and association-ends (in the contextual model).
- It provides standard operators on primitive data, tuples, and (parametrized) collections, and special operators to *iterate* over collections, such as `forAll`, `exists`, `select`, and `collect`.
- Collection can be *sets*, *bags*, *ordered sets* and *sequences*, and can be parametrized by any type, including other collection types.
- OCL provides two constants to represent *undefinedness*: `null` represents an unknown or undefined value, and `invalid` represents an error or exception.

SECUREUML: USAGE OF OCL

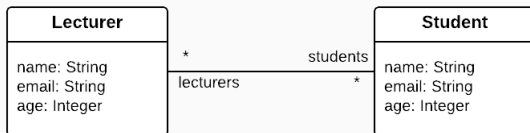
As mentioned, in SecureUML, authorization constraints are specified using the OCL. More specifically, the authorization constraints are specified depend on the information related to:

- the user who attempting to access the data,
- the object whose attributes are attempted to be accessed,
- and the objects between which the links are attempted to be accessed.

Using the OCL, we preserved the following keywords for these information, in respective:

- caller,
- self,
- ase_l and ase_r, where ase_l and ase_r represents the corresponding association-ends.

SECUREUML AUTHORIZATION CONSTRAINT (EXAMPLE)



Given an authorization constraint *every lecturer can only read their students' records*, it can be specified in OCL as follows:

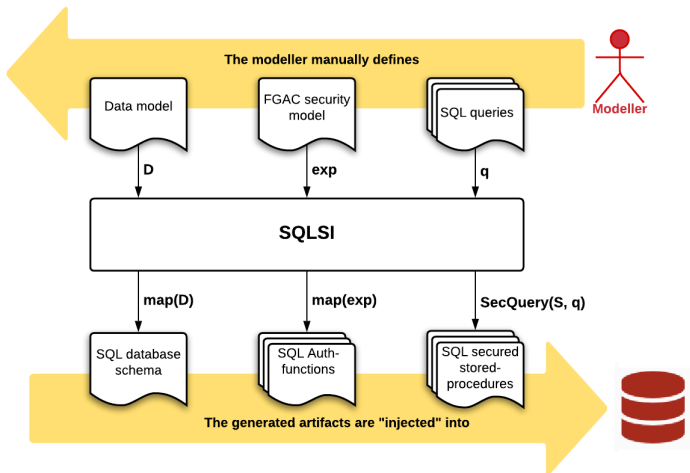
OCL AUTHORIZATION CONSTRAINT

```
caller.students→includes(self)
```

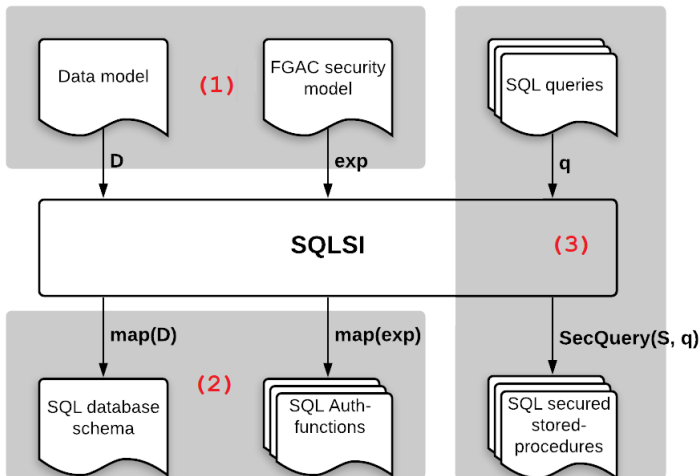
where the keyword caller represents the lecturer who is attempting to read and the keyword self represents the student whose record is attempted to be read.

A model-driven approach for FGAC Policies for SQL queries

A MODEL-DRIVEN APPROACH FOR ENFORCING FGAC POLICIES ON DATABASES: IN A NUTSHELL



A MODEL-DRIVEN APPROACH FOR ENFORCING FGAC POLICIES ON DATABASES: IN A NUTSHELL



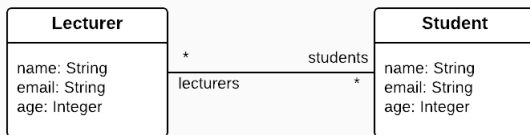
MODELLING DATA MODELS

Data models specify the resources to be protected.

Let $\mathcal{T}$ be a set of predefined types. A *data model* $\mathcal{D}$ is a tuple $\langle C, AT, AS \rangle$, where:

- C is a set of *classes* c .
- AT is a set of *attributes* at , $at = \langle atn, c, t \rangle$, where: atn is name of the attribute; c is the class of the attribute; and t is the type of the values of the attribute, with either $t \in \mathcal{T}$ or $t \in C$.
- AS is a set of *associations* as , $as = \langle asn, ase_l, c_l, ase_r, c_r \rangle$, where: asn is the name of the association; ase_l and ase_r are the ends of the association as ; c_l is the class of the objects at the association-end ase_l ; and c_r is the class of the objects at the association-end ase_r .

DATA MODELS (EXAMPLE)



- There are lecturers and students.
- Lecturers can teach students (they are their `students`), and students are taught by lecturers (they are their `lecturers`).
- No restriction is imposed regarding enrollment: a lecturer can teach many different students (or none), and a student can be taught by many different lecturers (or by none).
- Each lecturer and student have a name, an age and an email.

DATA MODELS (EXAMPLE)

According to our definition, the aforementioned data model can be defined as follows:

DATA MODEL

$$C = \{\text{Student}, \text{Lecturer}\},$$

$$AT = \{\langle \text{name}, \text{Student}, \text{String} \rangle, \langle \text{age}, \text{Student}, \text{Integer} \rangle, \\ \langle \text{email}, \text{Student}, \text{String} \rangle, \langle \text{name}, \text{Lecturer}, \text{String} \rangle, \\ \langle \text{age}, \text{Lecturer}, \text{Integer} \rangle, \langle \text{email}, \text{Lecturer}, \text{String} \rangle\}$$

$$AS = \{\langle \text{Enrollment}, \text{students}, \text{Student}, \text{lecturers}, \text{Lecturer} \rangle\}$$

OBJECT MODELS AND THE EVALUATION OF OCL EXPRESSIONS

Object models (also called scenarios) are instances of data models.

Let $\mathcal{D}$ be a data model. $\text{Exp}(\mathcal{D})$ denotes the set of OCL expressions whose contextual model is $\mathcal{D}$.

Let $\mathcal{O}$ be an instance of $\mathcal{D}$, and let e be an OCL expression in $\text{Exp}(\mathcal{D})$. Then, $\text{Eval}(\mathcal{O}, e)$ denotes the result of *evaluating* e in $\mathcal{O}$, according to the semantics of OCL.

MODELLING FGAC SECURITY MODELS

Let $\mathcal{D}$ be a data model, $\mathcal{D} = \langle C, AT, AS \rangle$. We denote by $\text{Act}(\mathcal{D})$ the following set of read-actions:

- For every attribute $at \in AT$, $\text{read}(at) \in \text{Act}(\mathcal{D})$.
- For every association $as \in AS$, $\text{read}(as) \in \text{Act}(\mathcal{D})$.

Then, a *security model* S for $\mathcal{D}$ is a tuple $S = \langle R, \text{auth} \rangle$, where: R is a set of roles, and

$$\text{auth}: R \times \text{Act}(\mathcal{D}) \rightarrow \text{Exp}(\mathcal{D})$$

is a function that assigns to each role $r \in R$ and each action $a \in \text{Act}(\mathcal{D})$ an authorization constraint $e \in \text{Exp}(\mathcal{D})$.

MODELLING FGAC SECURITY MODELS (REMARKS)

Let $\mathcal{D} = \langle C, AT, AS \rangle$ be a data model. Let $\mathcal{S} = \langle R, \text{auth} \rangle$ be an FGAC security model for $\mathcal{D}$. Let $\mathcal{O}$ be an object model of $\mathcal{D}$. Then,

- A user u with role $r \in R$ is authorized, according to $\mathcal{S}$, to read the value of an attribute $at = \langle atn, c, t \rangle$, $at \in AT$, of an object o , if and only if:

$$\text{Eval}(\mathcal{O}, \text{auth}(r, \text{read}(at))[\underline{\text{self}} \leftarrow o; \underline{\text{caller}} \leftarrow u]) = \text{true}.$$

- A user u with role $r \in R$ is authorized, according to $\mathcal{S}$, to read whether an association $as = \langle asn, ase_1, c_1, ase_r, c_r \rangle$, $as \in AS$, links two objects o_1 and o_r , if and only if:

$$\text{Eval}(\mathcal{O}, \text{auth}(r, \text{read}(as))[\underline{ase_1} \leftarrow o_1; \underline{ase_r} \leftarrow o_r; \underline{\text{caller}} \leftarrow u]) = \text{true}.$$

MODELLING FGAC SECURITY MODELS (EXAMPLE)

Given an FGAC security model that consists of one role, namely `Lecturer` and an FGAC policy: *every user with `Lecturer` role can read their students' age*:

FGAC SECURITY MODEL

```
R = {Lecturer},
auth(Lecturer, Student : age) =
    caller.students → includes(self)
```

MODELLING FGAC SECURITY MODELS (EXAMPLE)

Given an object model $\mathcal{O}$ where there is a student Hoang, a lecturer Alice, and Hoang is a student of Alice. Then, Alice is authorized to see the age of Hoang. Indeed:

FGAC SECURITY MODEL

$$\begin{aligned} & \text{Eval}(\mathcal{O}, \text{auth}(r, \text{read}(at))[\underline{\text{self}} \leftarrow \text{Hoang}; \underline{\text{caller}} \leftarrow \text{Alice}]) \\ &= \text{Eval}(\mathcal{O}, \text{Alice.students} \rightarrow \text{includes}(\text{Hoang})) \\ &= \text{true}. \end{aligned}$$

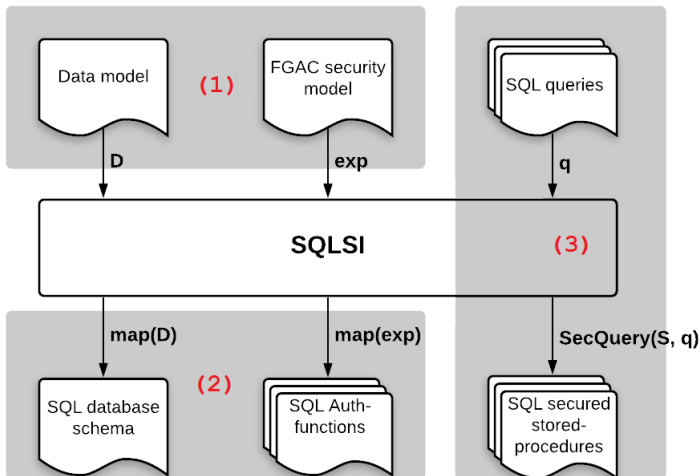
MODELLING FGAC SECURITY MODELS (EXAMPLE)

Given an object model where there is a student Hoang, a lecturer Bob, and Hoang is **not** a student of Bob. Then, Bob is **not** authorized to see the age of Hoang. Indeed:

FGAC SECURITY MODEL

```
Eval( $\mathcal{O}$ , auth( $r$ , read( $at$ ))[self  $\leftarrow$  Hoang; caller  $\leftarrow$  Bob])
= Eval( $\mathcal{O}$ , Bob.students  $\rightarrow$  includes(Hoang))
= false.
```

A MODEL-DRIVEN APPROACH FOR ENFORCING FGAC POLICIES ON DATABASES: IN A NUTSHELL



MAPPING FROM DATA MODELS TO DATABASE SCHEMA

As part of the previous work, we defined a mapping from data models to SQL database schema.

Let $\mathcal{D}$ be a data model. In what follows we denote by $\text{map}(\mathcal{D})$ the result of mapping data model $\mathcal{D}$ to SQL database schema.

The definition of $\text{map}(\mathcal{D})$ is recalled in Appendix A in the thesis manuscript.

MAPPING FROM DATA MODELS TO DATABASE SCHEMA (EXAMPLE)

SQL DATABASE SCHEMA

```
Student(Student_id, name, age, email),  
Lecturer(Lecturer_id, name, age, email),  
Enrollment(lecturers, students)
```

where `Student_id`, `Lecturer_id` are the primary keys whereas `lecturers`, `students` are the foreign keys of table `Student`, `Lecturer`, respectively.

MAPPING FROM OBJECT MODELS TO DATABASE INSTANCES

As part of the previous work, we defined a mapping from object models to SQL database instances.

Let $\mathcal{D}$ be a data model. Let $\mathcal{O}$ be an object model of $\mathcal{D}$. In what follows we denote by the function $\text{map}(\mathcal{O})$ the result of mapping object model $\mathcal{O}$ to SQL database instance.

The definition of $\text{map}(\mathcal{O})$ is recalled in Appendix B in the thesis manuscript.

CORRECTNESS IMPLEMENTATION OF OCL AUTHORIZATION CONSTRAINTS IN SQL STATEMENTS

Let $\mathcal{D}$ be a data model. Let $exp \in \text{Exp}(\mathcal{D})$ be a boolean expression. We say that $\text{map}(exp)$ is a *correct* implementation of exp in $\mathcal{D}$ if and only if:

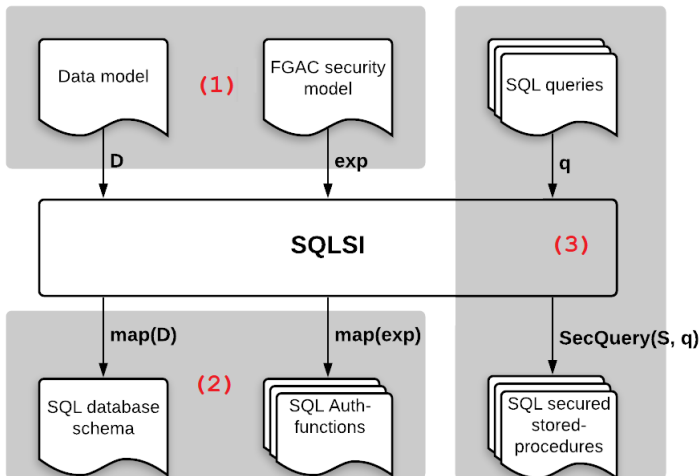
$$\text{Eval}(\mathcal{O}, exp) = \text{true} \iff \text{Exec}(\text{map}(\mathcal{O}), \text{map}(exp)) = \text{TRUE}$$

for every object model $\mathcal{O}$ of $\mathcal{D}$.

GENERATING SQL AUTHORIZATION FUNCTIONS FROM FGAC SECURITY MODELS

- For each protected resource p , an SQL function $\text{AuthFunc}(p)$ is generated to check that the access to p is authorized. Depending on the role r , executing this function involves calling the function $\text{AuthFuncRole}(p, r)$ as defined below.
- For each protected resource p , for each role r , an SQL function $\text{AuthFuncRole}(p, r)$ is generated that implements the authorization check required for a user with role r to be authorized to access p . This function includes the SQL correct implementation of the OCL authorization constraint $\text{auth}(r, \text{read}(p))$.

A MODEL-DRIVEN APPROACH FOR ENFORCING FGAC POLICIES ON DATABASES: IN A NUTSHELL



REWRITING SQL QUERIES

Given an FGAC security model $\mathcal{S}$ and an SQL query q , an SQL stored-procedure $\text{SecQuery}(\mathcal{S}, q)$ is generated. The procedure takes two arguments: the user attempting to execute q and his role.

The body of this stored-procedure comprises a list of temporary tables, corresponding to the list of conditions that need to be satisfied for that user, with that role, to be authorized to execute the query q , according to $\mathcal{S}$.

For example, consider the query of form:

SELECT at_1 FROM c WHERE $at_2 = value$

for a user u , with role r , to be authorized to execute this, the database system must check:

- For every object in c , $\langle u, r \rangle$ is authorized to read the value of at_2 .
- For every object in c that satisfies $at_2 = value$, $\langle u, r \rangle$ is authorized to read the value of at_1 .

REWRITING SQL QUERIES (CONT.)

More specifically, these temporary tables include the SQL authorization functions `AuthFunc()` generated by the FGAC security models. As a result, creating these temporary tables involves executing these authorization checks.

- If the execution of these functions return `TRUE`, meaning that u , with role r , is authorized to execute q , according to $\mathcal{S}$, then the final result of calling `SecQuery( $\mathcal{S}, q$ )` is equivalent to executing q .
- Otherwise, if any of the execution of these functions return `FALSE`, meaning that u , with role r , is **not** authorized to execute q , according to $\mathcal{S}$, then the computation stops and an error is signal.

THE INPUT SQL QUERIES (EXAMPLE)

Consider the following SQL query

```
SELECT COUNT(Student_id) FROM Student WHERE age > 18
```

For any lecturer l attempting to execute this query, the database system must check whether l is authorized to read the age of all the students.

THE GENERATED STORED-PROCEDURES (EXAMPLE)

```

1 CREATE PROCEDURE Query1(in caller varchar(250),
2 in role varchar(250))
3 BEGIN
4     DECLARE _rollback int DEFAULT 0;
5     DECLARE EXIT HANDLER FOR SQLEXCEPTION
6     BEGIN
7         SET _rollback = 1;
8         GET STACKED DIAGNOSTICS CONDITION 1
9         @p1 = RETURNED_SQLSTATE, @p2 = MESSAGE_TEXT;
10        SELECT @p1, @p2;
11        ROLLBACK;
12    END;
13    START TRANSACTION;
14    DROP TEMPORARY TABLE IF EXISTS TEMP1;
15    CREATE TEMPORARY TABLE TEMP1 AS (
16        SELECT * FROM Student WHERE
17            CASE auth_READ_Student_age(caller, role, Student_id)
18            WHEN 1 THEN age ELSE throw_error() END > 18
19    );
20    DROP TEMPORARY TABLE IF EXISTS TEMP2;
21    CREATE TEMPORARY TABLE TEMP2 AS (
22        SELECT Student_id AS Student_id FROM TEMP1
23    );
24    IF _rollback = 0
25    THEN SELECT COUNT(*) FROM TEMP2;
26    END IF;
27    END //
28    DELIMITER ;

```

THE KEY FEATURES

1. The enforcement mechanism leaves unmodified the underlying database, except for adding the stored-procedures that configure the FGAC-enforcement mechanism.
2. The FGAC policies and the database queries are kept independent of each other, except that they refer to the same underlying data model.
3. The enforcement mechanism can be automatically generated from the FGAC-policies, with appropriate mapping definition `map()` from OCL authorizations to SQL statements.

DRAWBACKS: THE PERFORMANCE OVERHEAD

- FGAC policies depend on the satisfaction of authorization constraints on the current state of the system.
- These authorization constraints must be checked at runtime.
- This causes a huge performance penalty.

```
1 CREATE TEMPORARY TABLE TEMP1 AS (  
2   SELECT * FROM Student WHERE  
3     CASE auth_READ_Student_age(caller, role, Student_id)  
4     WHEN 1 THEN age ELSE throw_error() END > 18  
5 );
```

INTELLIGENTLY ENFORCING FGAC POLICIES FOR SQL QUERIES

However, there are cases, in which, the authorization checks seem unnecessary:

- In case the authorization check can be proved to be trivially true.
- In case the authorization check can be proved to be satisfied, given an integrity constraint of the underlying data model.
- In case the authorization check can be proved to be satisfied, given the properties of the objects involved in the authorization request.

In this thesis, we propose a methodology to formally prove those cases and to optimize the generated stored-procedure.

Intelligently enforcing FGAC Policies for SQL queries

MOTIVATION

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of any student.

```
1 CREATE TEMPORARY TABLE TEMP1 AS (  
2   SELECT * FROM Student WHERE  
3     CASE auth_READ_Student_age(caller, role, Student_id)  
4       WHEN 1 THEN age ELSE throw_error() END > 18  
5 );
```

In this case, it seems that the check is unnecessary since it always return `TRUE`.

MOTIVATION

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of their own students.

In addition, consider the following database integrity constraints:
every lecturer teaches every student.

```
1 CREATE TEMPORARY TABLE TEMP1 AS (  
2   SELECT * FROM Student WHERE  
3     CASE auth_READ_Student_age(caller, role, Student_id)  
4       WHEN 1 THEN age ELSE throw_error() END > 18  
5 );
```

In this case, if the invariant holds, then the check seems unnecessary.

MOTIVATION

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of the students if he is the oldest lecturer.

In addition, consider the scenario where the user is indeed the oldest lecturer.

```
1 CREATE TEMPORARY TABLE TEMP1 AS (  
2   SELECT * FROM Student WHERE  
3     CASE auth_READ_Student_age(caller, role, Student_id)  
4       WHEN 1 THEN age ELSE throw_error() END > 18  
5 );
```

In this case, since the user is the oldest lecturer, then the check seems unnecessary.

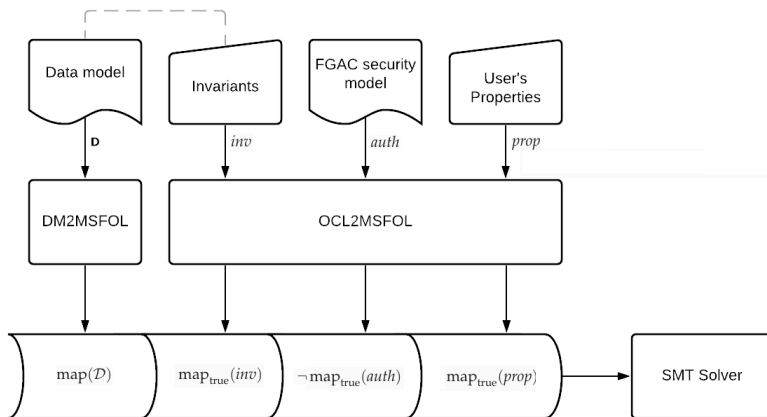
GENERAL APPROACH

We conduct the proof by reducing the problem to a satisfiability problem:

*Given a data model $\mathcal{D}$, can one find a model $\mathcal{O}$ of $\mathcal{D}$ in which a user u , with role r , is **not** authorized to access the resources p of object o , according to the FGAC security model S ?*

If there is no model that satisfies the above property, then for every $\mathcal{O}$ of $\mathcal{D}$, the user u , with role r , is always authorized to access the resources p of object o , according to the FGAC security model S .

GENERAL APPROACH

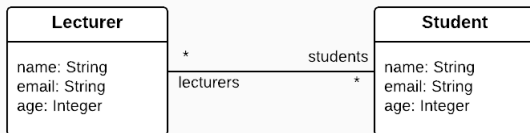


MAPPING FROM DATA MODELS TO MSFOL THEORIES

Let $\mathcal{D} = \langle C, AT, AS \rangle$ be a data model. $\text{map}(\mathcal{D})$ defines the MSFOL theory corresponding to $\mathcal{D}$. In a nutshell, this mapping contains:

- The sort Classifier, and two constant symbols, nullClassifier and invalClassifier.
- For every predefined type $t \in \mathcal{T}$, we create a sort t and two constant symbols, namely null t and inval t .
- For each class $c \in C$, a unary predicate $c : \text{Classifier} \rightarrow \text{Bool}$.
- For each attribute $at \in AT$, $at = \langle atn, c, t \rangle$, a function $atn.c : \text{Classifier} \rightarrow t$.
- For each association $as \in AS$, $as = \langle asn, ase_l, c_l, ase_r, c_r \rangle$, a binary predicate $asn : \text{Classifier} \times \text{Classifier} \rightarrow \text{Bool}$.
- The axioms constraining the meaning of the defined constants, predicates and functions.

MAPPING FROM DATA MODELS TO MSFOL THEORIES (EXAMPLE)



- There are lecturers and students.
- Lecturers can teach students (they are their `students`), and students are taught by lecturers (they are their `lecturers`).
- No restriction is imposed regarding enrollment: a lecturer can teach many different students (or none), and a student can be taught by many different lecturers (or by none).
- Each lecturer and student have a name, an age and an email.

MAPPING FROM DATA MODELS TO MSFOL THEORIES (EXAMPLE)

The mapping generates a MSFOL theory that contains:

- The sorts Classifier, Int and String, to represent objects, integer values and string values, respectively.
- The constant nullClassifier, invalClassifier and an axiom constraining that these two constants have different interpretation (and similarly, for sort Int and String):

$$\text{nullClassifier} \neq \text{invalClassifier}$$

$$\text{nullString} \neq \text{invalString}$$

$$\text{nullInt} \neq \text{invalInt}$$

MAPPING FROM DATA MODELS TO MSFOL THEORIES (EXAMPLE)

- For class `Lecturer`, the predicate `Lecturer( $x : \text{Classifier}$ )` and axioms constraining that the `nullClassifier` and `invalClassifier` are not of type `Lecturer`. In addition, an axiom constraining that a `Lecturer` object, cannot be a `Student` object.

$$\text{Lecturer}(\text{nullClassifier}) = \perp$$

$$\text{Lecturer}(\text{invalClassifier}) = \perp$$

$$\forall x : \text{Classifier}. \text{Lecturer}(x) \implies \neg \text{Student}(x)$$

- Similarly, for class `Student`.

$$\text{Student}(\text{nullClassifier}) = \perp$$

$$\text{Student}(\text{invalClassifier}) = \perp$$

$$\forall x : \text{Classifier}. \text{Student}(x) \implies \neg \text{Lecturer}(x)$$

MAPPING FROM DATA MODELS TO MSFOL THEORIES (EXAMPLE)

- For attribute `age` of `Lecturer`, the function

$$\text{age_Lecturer}(x : \text{Classifier}) : \text{Int}$$

and three axioms constraining that (i) it is invalid to get the age of a null object, (ii) it is invalid to get the age of an invalid object and (iii) the age of a lecturer cannot be invalid.

$$\text{age_Lecturer}(\text{nullClassifier}) = \text{invalidInt}$$

$$\text{age_Lecturer}(\text{invalidClassifier}) = \text{invalidInt}$$

$$\forall x : \text{Classifier}. \text{Lecturer}(x) \implies \text{age_Lecturer}(x) \neq \text{invalidInt}$$

- Similarly, the same applies for other attributes of other classes.

MAPPING FROM DATA MODELS TO MSFOL THEORIES (EXAMPLE)

- For association `Enrollment`, the binary predicate

$$\text{Enrollment}(x : \text{Classifier}, y : \text{Classifier})$$

and an axiom constraining the type of two association-ends.

$$\forall x : \text{Classifier}. \forall y : \text{Classifier}.$$

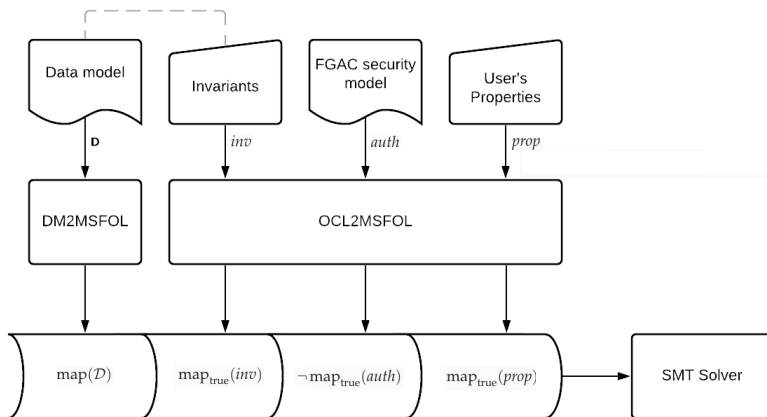
$$\text{Enrollment}(x, y) \implies \text{Lecturer}(x) \wedge \text{Student}(y)$$

MAPPING FROM OBJECT MODELS TO MSFOL INTERPRETATIONS

Let $\mathcal{D}$ be a data model. Let $\mathcal{O}$ be an object model of $\mathcal{D}$. $\text{map}(\mathcal{O})$ defines the MSFOL interpretation of the theory $\text{map}(\mathcal{D})$ that corresponds to the object model $\mathcal{O}$. In other words,

$$\text{map}(\mathcal{O}) \models \text{map}(\mathcal{D})$$

GENERAL APPROACH



MAPPING FROM OCL BOOLEAN EXPRESSIONS TO MSFOL FORMULAE

Let $\mathcal{D}$ be a data model and $exp \in \text{Exp}(\mathcal{D})$ be an OCL boolean expression. $\text{map}_{\text{true}}(exp)$ defines the MSFOL formulae of exp .

Moreover, the mapping satisfies the following property:

$$\text{map}(\mathcal{O}) \models \text{map}_{\text{true}}(exp) \iff \text{Eval}(\mathcal{O}, exp) = \text{true}$$

for any object model $\mathcal{O}$.

MOTIVATION (REVISITED)

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of any student.
- Denote the authorization constraint for reading the student's age by *auth*.
- In addition, two constants are created, caller — for the user who is accessing and self — the object which is accessed.
 - The user is of type `Lecturer`.
 - The object is of type `Student`.
- Indeed, the MSFOL theory:

$$\begin{aligned} &\text{map}(\mathcal{D}) \cup \neg \text{map}_{\text{true}}(\text{auth}) \\ &\quad \cup \text{map}(\text{caller}, \text{Lecturer}) \cup \text{map}(\text{self}, \text{Student}) \end{aligned}$$

is unsatisfiable.

MOTIVATION

In this case, we can safely remove the unnecessary check:

```
1 CREATE TEMPORARY TABLE TEMP1 AS (  
2   SELECT * FROM Student WHERE age > 18  
3 );
```

MOTIVATION (REVISITED)

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of their own students.

In addition, consider the following database integrity constraints:
every lecturer teaches every student.

- Similar to the previous case, denote by *auth* the authorization constraint for reading the student's age and caller and self two constants for the user and the object.
- Moreover, denote by *inv* the newly data invariants.
- Indeed, the MSFOL theory:

$$\begin{aligned} & \text{map}(\mathcal{D}) \cup \neg \text{map}_{\text{true}}(\text{auth}) \cup \text{map}_{\text{true}}(\text{inv}) \\ & \quad \cup \text{map}(\text{caller}, \text{Lecturer}) \cup \text{map}(\text{self}, \text{Student}) \end{aligned}$$

is unsatisfiable.

MOTIVATION (REVISITED)

In this case, to potentially reduce the performance overhead, we check for the satisfaction of the data invariants before executing the authorization checks.

- If the invariant holds, then the checks can be removed.
- Otherwise, we execute the authorization checks as are.

```

1  IF (/* the SQL correct implementation of the invariant */)
2  THEN
3      CREATE TEMPORARY TABLE TEMP1 AS (
4          SELECT * FROM Student WHERE age > 18
5      );
6  ELSE
7      CREATE TEMPORARY TABLE TEMP1 AS (
8          SELECT * FROM Student WHERE
9              CASE auth_READ_Student_age(caller, role, Student_id)
10                 WHEN 1 THEN age ELSE throw_error() END > 18
11      );

```

MOTIVATION (REVISITED)

Consider the following FGAC security model:

- There is only one role, namely `Lecturer`.
- Every lecturer is authorized to read the age of the students if he is the oldest lecturer.
- Similar to the previous case, denote by *auth* the authorization constraint for reading the student's age and caller and self two constants for the user and the object.
- Moreover, denote by *prop* the properties of the user.
- Indeed, the MSFOL theory:

$$\begin{aligned} \text{map}(\mathcal{D}) \cup \neg \text{map}_{\text{true}}(\text{auth}) \cup \text{map}_{\text{true}}(\text{prop}) \\ \cup \text{map}(\text{caller}, \text{Lecturer}) \cup \text{map}(\text{self}, \text{Student}) \end{aligned}$$

is unsatisfiable.

MOTIVATION (REVISITED)

Similar to the previous case, to potentially reduce the performance overhead, we check for the satisfaction of the user property before executing the authorization checks.

- If the property holds, then the checks can be removed.
- Otherwise, we execute the authorization checks as are.

```

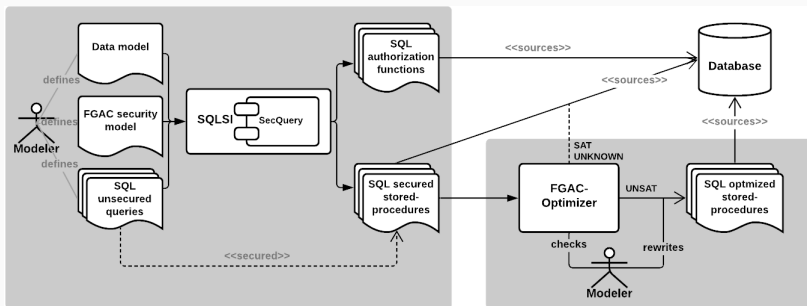
1  IF (/* the SQL correct implementation of the user property */)
2  THEN
3      CREATE TEMPORARY TABLE TEMP1 AS (
4          SELECT * FROM Student WHERE age > 18
5      );
6  ELSE
7      CREATE TEMPORARY TABLE TEMP1 AS (
8          SELECT * FROM Student WHERE
9              CASE auth_READ_Student_age(caller, role, Student_id)
10                 WHEN 1 THEN age ELSE throw_error() END > 18
11      );

```

TOOL SUPPORT

- SQLSI: To automatically enforce FGAC policies for SQL queries.
 - It takes a data model, a security model and a collection of SQL queries as input.
 - It returns a collection of SQL authorization functions and of SQL stored-procedures.
- FGAC-Optimizer: To formally prove that an authorization check is unnecessary.
 - It takes a data model, a security model and a database configuration (including information about the user, role, resource to be accessed, invariants, properties) as input.
 - It returns either SAT, UNSAT or UNKNOWN.

TOOL SUPPORT



SOME PRELIMINARY EXPERIMENTAL RESULTS

A non-trivial case study is conducted using a fixed data model, but different scenarios, FGAC policies, users, roles and queries.

- For all generated MSFOL theories, the SMT solvers return the expected result in less than 1 second.
- In the suitable scenarios, the execution-time of the optimized stored-procedures reduces significantly. In some cases, the calculated speed-up is over 100 times.

Conclusions

CONCLUSIONS

- Recently, a model-driven approach for enforcing FGAC policies when executing SQL queries has been proposed.
 - $\text{SecQuery}(S, q)$ returns a secured stored-procedure that implements all the authorization checks that are required to comply with security model S .
 - When execute at run-time, for any user u and any role r , if $\langle u, r \rangle$ is authorized to execute q , then calling $\text{SecQuery}(S, q)$ will result the same as executing q . Otherwise, an error will be signalled.
 - Not surprisingly, performing FGAC authorization checks at run-time results in performance loss.

CONCLUSIONS

- In this thesis, we propose a methodology to detect unnecessary checks and then to remove those checks to optimize the performance.
 - By mapping the related artifacts to FOL, those checks can be shown to be unnecessary using an automatic theorem prover, namely the SMT Solver.
 - Once proven, $\text{SecQuery}(\mathcal{S}, q)$ can be rewritten, take into account the newly obtained information.
 - As a result, in our case study, the execution-time has reduced significantly.

Thank you very much!